# Crack AI: Immersive 3D Automated Candidate Interview System

## 1. Executive Summary

The **AI Interviewer** is a cutting-edge recruitment technology that leverages 3D environments, real-time WebSockets, and LLMs to deliver automated technical interviews. This is unlike a video conference solution that puts the candidate in an office where they can talk to an AI using voice, text, and an interactive whiteboard.

## 2. Technical Architecture

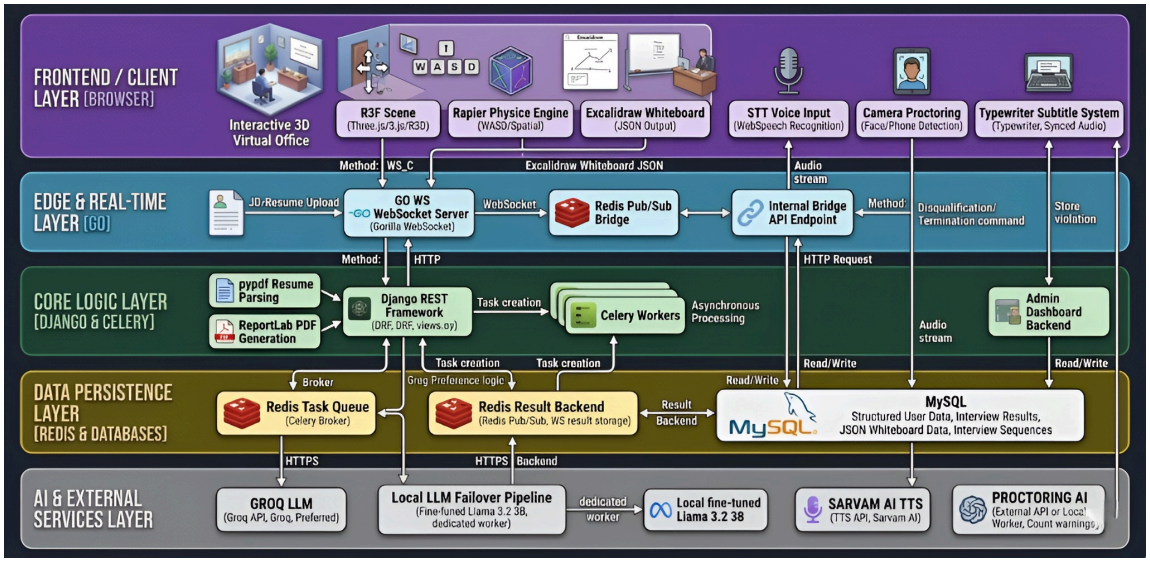
### 2.1 Component Stack

| Layer | Technologies |
|---|---|
| **Frontend** | React, Vite, TypeScript, Three.js, React Three Fiber (R3F), Rapier Physics, Excalidraw |
| **Real-Time Layer** | Go (Golang), Gorilla WebSockets, Redis Pub/Sub |
| **Backend Core** | Python, Django, Django REST Framework (DRF) |
| **Task Processing** | Celery, Redis (Broker & Result Backend) |
| **AI / ML** | Groq (Llama 3), Local Llama 3.2 3B (Fine-tuned), Sarvam AI (TTS) |
| **Monitoring** | OpenCV/TensorFlow (Proctoring), Chart.js (Admin Dashboard) |
| **Database** | MySQL, Redis |
| **Infrastructure** | Docker, Docker Compose |

## 2.2 Data Flow Logic

1. **Frontend (React)** receives user input data(Voice/3D position/Whiteboard data).
2. **Go WebSocket Server** is a high-concurrent gateway, linking frontend events to the backend.
3. **Django/Celery** is used for heavy AI logic processing, including Evaluation/PDF Generation, in an asynchronous manner.
4. **Redis Pub/Sub** is used for communication between the Go layer and Python layer.

## 2.3 Architecture Diagram




# 3. Core Implementation Modules

## 3.1 Immersive 3D Environment

- **Spatial Interaction:** Built with React Three Fiber, candidates navigate the office using WASD keys. Rapier Physics handles collision detection to maintain immersion.
- **Synchronized Media:** AI responses are converted to speech via **Sarvam AI**. The frontend uses a typewriter subtitle system synced to the audio duration for "human-like" delivery.

## 3.2 Multimodal Interview Engine

- **Dynamic Questioning:** Upon JD/Resume upload, a question sequence is generated (Self-Intro, Project-specific, DSA, etc.).
- **Voice Integration:** Uses WebSpeech API for Real-time STT (Speech-to-Text).
- **Visual Assessment:** For the DSA/Architecture phase, candidates use **Excalidraw**.

### 3.3 AI Inference Pipeline

- **Hybrid Intelligence:** The system prioritizes **Groq** for ultra-fast inference. If cloud services are unavailable, it fails over to a **Local Llama 3.2 3B** model running on dedicated Celery workers.
- **Contextual Awareness:** The AI "reads" the resume in real-time, allowing it to provide hints or pivot questions based on the candidate's previous answers.

### 3.4 Automated Proctoring & Integrity

- **Computer Vision (CV):** A background camera feed monitors the candidate.
- **Detection Logic:** Using pre-trained models, the system detects multiple persons or mobile devices.
- **Disqualification Protocol:** If the system triggers 5 warnings, the interview session is automatically terminated and the candidate is marked as disqualified in the database.

## 4. Administrative Features

### 4.1 HR Dashboard

- **Candidate Tracking:** A comprehensive view of all interview attempts.
- **Data Visualization:** Interactive charts (via Chart.js) visualize score distributions, skill gaps, and performance metrics across the candidate pool.
- **Reporting:** Generates a professional PDF via ReportLab detailing every question, answer, individual score, and qualitative AI feedback.

## 5. Challenges Faced

1. **3D Optimization:** Optimized high poly Blender models for a smooth 60 FPS experience in a browser environment by employing texture baking and gltfjsx.
2. **Real-Time Latency:** To overcome Django's synchronous limitations, we built a custom Go-based WebSocket server bridged via Redis Pub/Sub for high-speed, concurrent messaging.
3. **Visual-to-Text Mapping:** We developed a JSON-to-Text parser that translates Excalidraw's coordinate data into structured descriptions, allowing text-based LLMs to "see" and grade technical diagrams.
4. **AV Synchronization:** We achieved "human-like" interaction by dynamically calculating typewriter speeds based on the exact millisecond duration of the Sarvam AI audio clips.
5. **Performance Bottlenecks:** We offloaded the heavy Computer Vision proctoring logic to background Web Workers to prevent the camera feed from lagging the 3D environment's rendering.